

Programación

EXAMEN PRÁCTICO – EXTRAORDINARIA TEMAS 1-7

(17/06/2026)



LEE ATENTAMENTE LAS SIGUIENTES INSTRUCCIONES ANTES DE EMPEZAR:



- Si estás leyendo esto es porque ya te has descargado el enunciado del examen. Así que, desde este momento **ya puedes desconectar el cable Ethernet de tu equipo.**
- Cuando termines de resolver el examen, **avisa a tu profe** para conectar de nuevo el cable Ethernet y realizar la entrega.
- **Sube el proyecto Java comprimido (.zip) a la entrega de AULES disponible.**

PARTE 1: Configuración del entorno

1. Crea un nuevo proyecto Java (Maven) con IntelliJ -o el IDE que utilices-. Llámalo "EXTRAORDINARIA".
2. Crea en el proyecto dos paquetea nuevos llamados "copiones" y "hogueras" para ir añadiendo las clases correspondientes al problema planteado.

PARTE 2: Resolución de problemas

Programa en Java la solución al siguiente problema. Usa el proyecto que te acabas de crear en el apartado anterior.

(3,5p) PROBLEMA 1: DETECTANDO COPIONES

La profe está corrigiendo los exámenes finales de programación. Ha decidido digitalizar las respuestas de cada estudiante asignándoles un **código numérico** único que representa el "estilo" de resolución de cada examen. Al empezar a corregir, sospecha fuertemente que algunos alumnos se han copiado porque han escrito exactamente las mismas respuestas. La profe lo tiene claro: **si dos o más códigos de examen son exactamente iguales, significa que ha habido copiones.**

ENTRADA: la entrada del programa consta de una línea con números enteros positivos en formato **N N N N [...]** que representan los códigos generados para cada examen de la clase. Utiliza la siguiente expresión regular:

String REGEX = "^\\d{1,2}(\\d{1,2})*\$";

SALIDA: el programa mostrará el número total de exámenes copiados.

```
> Introduce códigos anticopia generados:
1 2 1 2 1
-----
Número total de exámenes copiados: 3

> Introduce códigos anticopia generados:
7 7 7 7 7 12 14
-----
Número total de exámenes copiados: 4

> Introduce códigos anticopia generados:
10 4 8 5 11 3
-----
Sin copias :)
```

El programa **debe controlar que el número total de exámenes no supere los 20 alumnos que tiene en clase ni tampoco se puedan generar códigos anticopia > 20.** En caso de error, el programa debe volver a insistir al usuario hasta que introduzca valores correctos.

```
> Introduce códigos anticopia generados:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
ERROR. El número de exámenes introducidos supera el límite máximo (20 estudiantes)
> Introduce códigos anticopia generados:
|
```

```
> Introduce códigos anticopia generados:
aaa
Formato no válido.
> Introduce códigos anticopia generados:
|
```

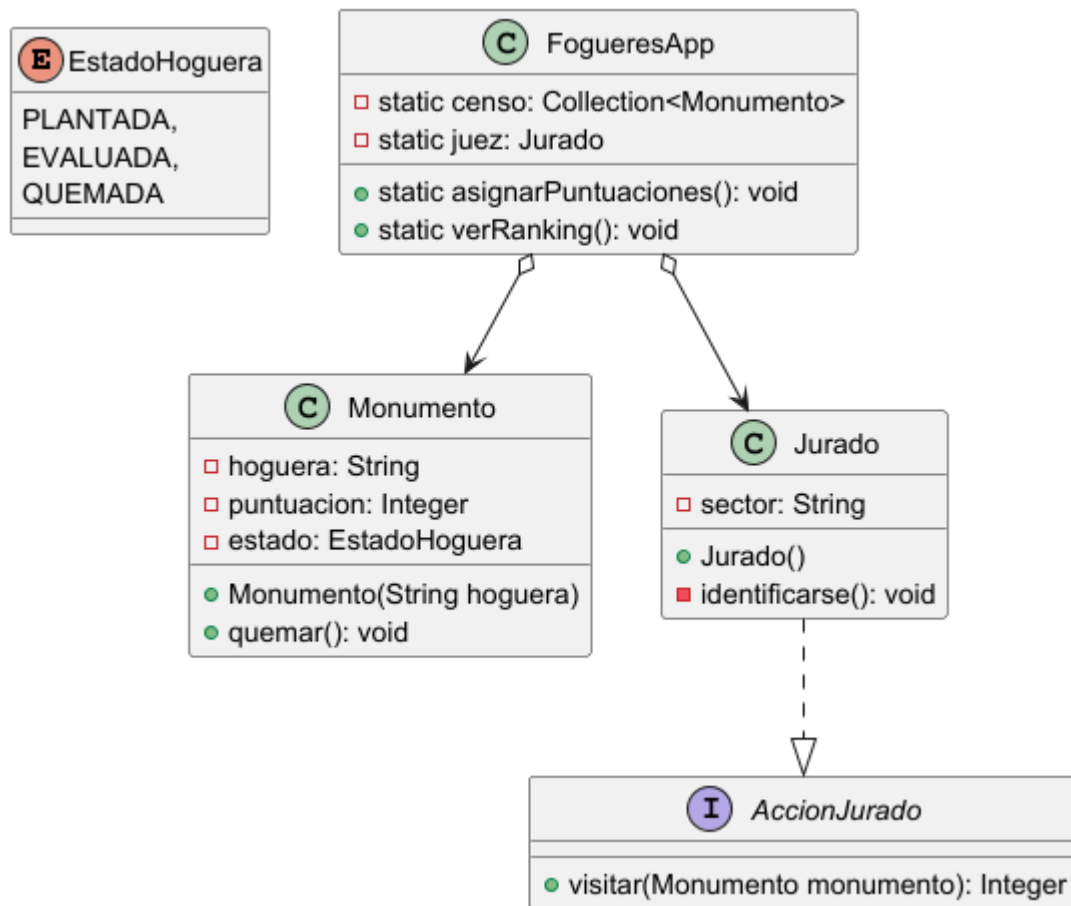
```
> Introduce códigos anticopia generados:
10 25 4 5 11 3
ERROR. No puede existir un código=25 (máx 20 estudiantes)
> Introduce códigos anticopia generados:
|
```

(6,5p) PROBLEMA 2: LAS HOGUERAS PREMIADAS

La *Federació de les Fogueres de Sant Joan* necesita una *app* para centralizar las votaciones del jurado y que no se traslapen las puntuaciones que van otorgando a cada monumento tras su visita, ya que últimamente está habiendo bastante follón porque algunas comisiones acusan de pucherazo al concejal de fiestas del Ayuntamiento.



El diagrama que ha pensado el informático del consistorio para la futura *app* es el siguiente:



Realiza un proyecto en *Java* para implementar la aplicación dada. Tienes libertad para modificar cualquier parte de la estructura, siempre que respetes la funcionalidad que se pide a continuación. Usa *Lombok* si te resulta útil. No se permite el uso de funciones *lambdas* y *streams*.

Condiciones para la construcción de las clases y otras cosas a tener en cuenta (LEE BIEN)

- (1p) Crea la estructura de clases** correspondiente al diagrama *UML* proporcionado.
- (0,5p) Carga manual de datos:** al arrancar, haz que el programa llene el *censo* de hogueras con **5 monumentos** para tener datos de prueba.
 - Asigna **0 puntos** y *estado* **PLANTADA** por defecto.
 - Muestra el contenido del *censo* para comprobar que se han insertado correctamente.

```

*** BIENVENIDO A LA APP DEL JURADO - HOGUERAS 26' ***
[
Monumento{hoguera='Sèneca-Autobusos', puntuacion=0, estado=PLANTADA},
Monumento{hoguera='Diputació-Renfe', puntuacion=0, estado=PLANTADA},
Monumento{hoguera='Carolinas Altas', puntuacion=0, estado=PLANTADA},
Monumento{hoguera='La Ceràmica', puntuacion=0, estado=PLANTADA},
Monumento{hoguera='Port d'Alacant', puntuacion=0, estado=PLANTADA}]
    
```

- c) **(1p) Regístrate como jurado.** Crea un objeto estático de tipo *Jurado* que emulará al usuario registrado que va a manejar la *app*. Usa el método privado *identificarse()* para pedir el *sector* al que perteneces para terminar de construir el objeto:

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:0
*** BIENVENIDO A LA APP DEL JURADO - HOGUERAS 26' ***
Introduce el sector al que perteneces:
hosteleria
Jurado del sector hosteleria registrado con éxito.
```

- d) **(1,75p)** Lo siguiente que haremos será imitar que el *Jurado* está visitando todas las hogueras para puntuarlas con el método *asignarPuntuaciones()*.

Recorreremos el *censo* de monumentos, y para cada uno, el juez lanzará el método *visitar(Monumento monumento)*. Este imprimirá “*Juez visitando y puntuando con un [puntuación random] a la hoguera [nombre hoguera]*”. La *puntuación* que se asigna será aleatoria, con un número entre 1 y 10.

```
Juez visitando y puntuando con un 7 a la hoguera Sèneca-Autobusos
Juez visitando y puntuando con un 5 a la hoguera Diputació-Renfe
Juez visitando y puntuando con un 2 a la hoguera Carolinas Altas
Juez visitando y puntuando con un 4 a la hoguera La Ceràmica
Juez visitando y puntuando con un 10 a la hoguera Port d'Alacant
```

Esta puntuación se la devolveremos al método *asignarPuntuaciones()* para que actualice la *puntuación* y ponga el *estado* = *EVALUADO* del monumento que estemos recorriendo. **Vuelve a mostrar el contenido del *censo* para comprobar que se han actualizado correctamente.**

```
Juez visitando y puntuando con un 7 a la hoguera Sèneca-Autobusos
Juez visitando y puntuando con un 5 a la hoguera Diputació-Renfe
Juez visitando y puntuando con un 2 a la hoguera Carolinas Altas
Juez visitando y puntuando con un 4 a la hoguera La Ceràmica
Juez visitando y puntuando con un 10 a la hoguera Port d'Alacant
[
Monumento{hoguera='Sèneca-Autobusos', puntuacion=7, estado=EVALUADA},
Monumento{hoguera='Diputació-Renfe', puntuacion=5, estado=EVALUADA},
Monumento{hoguera='Carolinas Altas', puntuacion=2, estado=EVALUADA},
Monumento{hoguera='La Ceràmica', puntuacion=4, estado=EVALUADA},
Monumento{hoguera='Port d'Alacant', puntuacion=10, estado=EVALUADA}]
```

- e) **(1,75p)** Una vez evaluadas todas las hogueras, la *app* debe mostrar un ranking de los 3 monumentos que más puntuación han obtenido. En caso de empate, aparecerán por orden alfabético.

```
Ranking de Hogueras:  
#1. Port d'Alacant con 10 puntos  
#2. Sèneca-Autobusos con 7 puntos  
#3. Diputació-Renfe con 5 puntos
```

- f) **(0,5p)** Quema todas las hogueras censadas con el método *quemar()* (modifica el **estado** = **QUEMADA**) y vete de vacaciones.

```
El pirotécnico de Carolinas Altas tira la traca final y la hoguera se quema.  
El pirotécnico de La Ceràmica tira la traca final y la hoguera se quema.  
El pirotécnico de Sèneca-Autobusos tira la traca final y la hoguera se quema.  
El pirotécnico de Diputació-Renfe tira la traca final y la hoguera se quema.  
El pirotécnico de Port d'Alacant tira la traca final y la hoguera se quema.
```